

VMC Control Center — Documento de Continuidad del Proyecto

Última actualización: 6 de mayo de 2026

1. CONTEXTO GENERAL

Propietario: Daniel (daniel_2896@hotmail.com) **Máquina:** M02-UK-BC8A (código BC8A08520A50) **Ubicación:** Cuernavaca, Morelos, México **Plataforma actual:** csmology.com (proveedor chino) **Objetivo final:** Sistema 100% independiente del servidor chino

2. HARDWARE IDENTIFICADO

2.1 Tableta/SBC principal

- **SoC:** Rockchip RK3568 (ARM Cortex-A55 quad-core)
- **Android:** 11 (SDK 30)
- **Build:** HW-RK3568-V3.011.01R-20250220-ZH
- **Almacenamiento:** 32 GB eMMC (7.25 GB usados)
- **WiFi:** 2.4 GHz, IP: 192.168.0.8, MAC: 60:ff:9e:34:41:dd
- **Red WiFi:** WIFI_9721, Gateway: 192.168.0.1
- **4G/LTE:** Módulo mPCIe (respaldo celular, etiqueta "CONTROL COM5")
- **Puertos seriales nativos:** /dev/ttyS0 a /dev/ttyS6 (7 puertos)
- **Puertos USB-Serial:** /dev/ttyUSB2, /dev/ttyUSB3

2.2 Dispensador (motores que despachan productos)

- **Tipo:** SH (尚和 / ShangHe)
- **Puerto:** /dev/ttyS4
- **Baud rate:** 9600
- **Protocolo:** Serial propietario SH (bytes descifrados — ver sección 5)
- **RunMode:** 1

2.3 Billetero (aceptador de billetes)

- **Modelo:** ICT 524TAO-A/V+M
- **Versión firmware:** 0126

- **Número serial:** 250600011524
- **Puerto:** /dev/ttyS1 (vía adaptador Wafer MDB)
- **Protocolo:** MDB level 1
- **Baud rate:** 9600
- **Tipo MDB:** Wafer
- **Denominaciones bloqueadas:** \$18, \$90, \$180, \$360 MXN
- **Modo de cambio:** 0 (sin devolución de cambio)

2.4 Cashless (pagos sin efectivo)

- **Modelo:** DMX-2011
- **Fabricante:** NYX
- **Versión:** 0100
- **Serial:** 331225106955
- **Protocolo:** MDB level 3
- **Habilitado:** Sí

2.5 Fuente de poder

- **Marca:** DOZWAIT
- **Modelo:** ER3243-D-OH01-P02

2.6 Pantalla

- **Board LCD:** YFDR032056 (fecha 2025-09-01)
- **Conexión:** Cable flex al RK3568

3. SOFTWARE ACTUAL EN LA MÁQUINA

3.1 App principal

- **Paquete:** com.csm.vending (csmVending)
- **Tamaño:** ~28 MB (APK extraído: 12.6 MB comprimido)
- **Ruta APK:** /data/app/~~CDtCWSliHdBJgmnmmnnYOw==/com.csm.vending-GtIz31ftc-SiFeu7jdrE8w==/base.apk
- **Versión config:** 02.00.52 (AfcConfig), 02.00.48 (ModuleConfig)

3.2 Otras apps relevantes

App	Función
ComAssistant (238 KB)	Herramienta de comunicación serial — COMA:/dev/ttyUSB3, COMB:/dev/ttyUSB2, ambas a 9600 baud
Termux (1.47 MB)	Terminal Linux (instalada, funcional)
ES Explorador PRO (32 MB)	Explorador de archivos
Firefox (40 MB)	Navegador web
Editor de código (58 MB)	Editor de texto

3.3 Carpetas clave en /sdcard/

- **csmConfig/** — Configuración de la máquina (propiedades de hardware)
- **csmLog/** — Logs de operación (contienen bytes seriales reales)
- **csmAds/** — Publicidad de la plataforma
- **backups/** — RespalDOS
- **imstlife/** — Datos adicionales

3.4 Conexiones de red del sistema actual

Servicio	URL	Protocolo
API REST	https://api.vmc002.csmology.com	HTTPS
WebSocket	wss://ws.vmc002.csmology.com:443/vmcSocket	WSS
Auth	POST /auth/devAccessToken	HTTPS
Messaging	RabbitMQ AMQP v5.9.0	AMQP

4. RESULTADOS DE LA INGENIERÍA INVERSA DEL APK

4.1 Estructura del APK

- **DEX:** classes.dex (8.7 MB) + classes2.dex (1 MB)
- **Librerías nativas:**
 - libserial_port.so (android-serialport-api, todas las arquitecturas)
 - libyy_serial_port.so (armeabi-v7a)
 - libOURIDR.so (RFID reader)
 - libOURMIFARE.so (MIFARE NFC)

4.2 Clases clave del dispensador SH

CDispenserDev_SH	- Controlador principal del dispensador
CDispenserDev_SH_1	- Variante 1
CDispenserDev_SH_2	- Variante 2
CDispenserDev_SH_YAxis	- Control del eje Y
CDispenserDev_SH_YAxisSidePick	- Recogida lateral
CDispenserStruct_SH	- Estructuras de datos del protocolo
CDispenserSH_Struct	- Struct del protocolo SH
CDispenserController	- Controlador principal
CDispenserController_HAL	- Capa de abstracción de hardware
CDispenserEventListener	- Listener de eventos
DispenserCtrlConfig	- Configuración del controlador

4.3 Clases clave MDB/Billetero

SerialPortOpt_WaferMDB	- Operaciones serial Wafer MDB
SerialPortOpt_Csm	- Operaciones serial CSM
SerialPortReaderOpt_Csm	- Lector serial
CHALCashlessStruct	- Struct de pagos cashless
CHALBillRouteType	- Tipos de ruta de billetes
CBillDepositController_HAL	- Controlador de depósito de billetes
CCashlessControllerDev_MDB	- Controlador cashless MDB
BillRecyclerDev_MDB	- Dispositivo reciclador de billetes

4.4 Bridge JavaScript ↔ Android

javascript

```
window.external_Android.MFC_SendCmd(cmd)           // Enviar comando
window.external_Android.MFC_SendShellCmd(cmd)       // Comando shell
window.external_Android.JsCallFunc(cmd, para)       // Llamar función
window.external_Android.call(cmd, para, callback)   // Llamada con callback
window.external_Android.openWindow(cmd, para)       // Abrir ventana
window.external_Android.closeWindow()              // Cerrar ventana
window.external_Android.MFC_GetCurFrame(cmd)       // Obtener frame
window.external_Android.MFC_PreviewPhoto()         // Preview foto
window.external_Android.setWindowFeatures(para)     // Configurar ventana
```

4.5 UI del cliente

- Framework: layui + jQuery 3.3.1
- Entry point: file:///android_asset/csmui/start/index.html
- Multilenguaje: i18n (configurado en español: System.lang=es)
- Carrito: límite configurable (Transaction.cart.limit=3)
- Archivos HTML: index.html, step.html, backer.html, dispenser-settings.html, etc.

4.6 Base de datos local (SQLite)

sql

```
-- Tabla de detalle de órdenes
"order_detail" (
  "_id" INTEGER PRIMARY KEY AUTOINCREMENT,
  "TOP_ORDER_NO" TEXT NOT NULL,
  "CURRENCY" TEXT,
  "GOODS_CODE" TEXT,
  "GOODS_NAME" TEXT,
  "ORDER_NUM" INTEGER,
  "SUCCESS_NUM" INTEGER,
  "FAILURE_NUM" INTEGER,
  "SINGLE_AMOUNT" REAL,
  "TOTAL_AMOUNT" REAL,
  "SHELF_NUMS" TEXT,
  "DELIVER_CODES" TEXT
);

-- Tabla de registro de transacciones
"trade_record" (
  "_id" INTEGER PRIMARY KEY AUTOINCREMENT,
  "ORDER_ID" TEXT NOT NULL,
  "TOP_ORDER_NO" TEXT,
  "TRACK_DATA" TEXT,
  "NUM" INTEGER,
  "PRICE" REAL NOT NULL,
  "TOTAL_AMOUNT" REAL NOT NULL,
  "TRACK_NO" INTEGER,
  "SHELF_NUM" TEXT,
  "GOODS_CODE" TEXT,
  "PAY_WAY" INTEGER,
  "PAY_TYPE" TEXT,
  "PAY_BILL_NO" TEXT,
  "STATUS" INTEGER
);
```

5. PROTOCOLO SERIAL SH — COMPLETAMENTE DESCIFRADO

5.1 Formato de trama

AA [LEN] [CMD] [DATA...] BB

AA = Byte de inicio (siempre)

LEN = Longitud de los datos
CMD = Código de comando
DATA = Datos variables
BB = Byte de fin (siempre)

5.2 Comando de despacho (VERIFICADO con log real)

ENVIAR: AA 06 01 [CABINET] [ROW_X] [COL_Y] BB
RECIBIR: AA 0B 01 02 [CABINET] [RESULT] 00 00 00 00 BB

Ejemplo real del log (test de slot #101, x=1, y=1):

Enviar: AA 06 01 01 01 0C BB

Recibir: AA 0B 01 02 01 01 00 00 00 00 BB → "配货成功" (despacho exitoso)

Donde:

CABINET = Número de gabinete (01 = principal)

ROW_X = Fila del producto (01-based)

COL_Y = Columna del producto (01-based, 0x0C = columna 12)

RESULT = 01 = éxito, otros = error

5.3 Heartbeat / Status polling (cada ~5 segundos)

RECIBIR: AA 06 03 [SLOT1] [SLOT2] ... [SLOTN] BB

Ejemplo real:

AA 06 03 01 01 01 01 01 01 01 01 01 01 01 BB

Donde cada byte de slot:

01 = Slot OK / disponible

00 = Slot vacío o error

5.4 Parámetros de la prueba de despacho (formato JavaScript)

json

```

{
  "cmd": "test.dispenser.deliveryTest",
  "param": {
    "devShelvesBean": {
      "cabinetNumber": 1,
      "deep": 15,
      "shelfIndex": 1,
      "shelfNum": "#101",
      "shelfType": 1,
      "x": 1,
      "y": 1
    }
  }
}

```

5.5 Numeración de slots

- Formato: #XYY donde X=gabinete, YY=número secuencial
- Ejemplo: #101 = Gabinete 1, Slot 01
- shelfNum mapea a coordenadas (x, y) para el comando serial

6. PROTOCOLO MDB — COMANDOS VERIFICADOS

6.1 Cashless polling

Enviar: 0x12 (COMMAND_POLL a CASHLESS)

Respuesta ACK: aceptado

Respuesta NAK: 4646200D0A (rechazado)

6.2 Bill Validator

COMMAND_BV_RESET	– Reset del validador
COMMAND_BV_ACTIVE	– Activar
COMMAND_BV_JUST_RESET	– Just reset
COMMAND_BV_ACCEPT_DISABLE	– Deshabilitar aceptación
COMMAND_ACK	– Confirmación
COMMAND_NAK	– Rechazo

7. PLATAFORMA WEB (csmology.com) — ENDPOINTS VERIFICADOS

7.1 Autenticación

- **Frontend:** <https://a.vmc002.csmology.com> (Vue 2 + Vuex + Element UI)
- **API base:** <https://api.vmc002.csmology.com>
- **Auth:** JWT HS512 Bearer token, ~2 horas de expiración
- **Login:** POST /auth/login con RSA-encrypted password + captcha (OBLIGATORIO)
- **RSA Public Key:**
MFwwDQYJKoZIhvcNAQEBBQADSwAwSAJBAN378k3RiZHWx5AfJqdH9xRNBmD9wG
D2iRe41HdTNF8RUhNnHit5NpMNtGL0NPTSSPjjI1kJfVorRvaQerUgkCAwEAAQ==

7.2 API Endpoints verificados

GET /auth/code	- Obtener captcha
GET /auth/info	- Info del usuario
POST /auth/login	- Login (username, password RSA, code, uuid)
GET /api/menus/build	- Menú (devuelve LISTA, no objeto)
GET /api/terminal?sort=createDate,desc&page=0&size=100	- Máquinas
GET /api/devGroup	- Grupos de dispositivos
GET /api/devCompStatus?terminalCode={code}	- Estado de componentes
POST /api/devCommand	- Enviar comando {terminalCode, control}
GET /api/goods	- Productos
GET /api/goodsType	- Tipos de producto
GET /api/additionalFee	- Tarifas adicionales
GET /api/commodityOrder?page=0&size=200&sort=createDate,desc	- Pedidos
GET /api/replenishmentOperations	- Reabastecimiento (puede dar 400)
GET /api/adResource	- Recursos publicitarios
GET /api/userTerminalAccess	- Accesos de usuario
GET /api/VMCConfigBase	- Configuración base
POST /api/statistics/getSaleCount	- Estadísticas de ventas

7.3 Comandos de control remoto

0100 = Normal Service (sin confirmación)
0101 = Out of Service (peligroso)
0301 = Shutdown (peligroso)
0302 = Reboot (verificado OK)
0307 = Restart Software (sin confirmación)
0401 = Upgrade (peligroso)

7.4 Endpoint de estadísticas — Formato EXACTO del request

json

POST /api/statistics/getSaleCount

Body:

```
{
  "timePeriodList": [
    {"startTime": "2026-04-24T00:00:00-06:00", "endTime": "2026-04-24T23:59:59-06:00"},
    {"startTime": "2026-04-01T00:00:00-06:00", "endTime": "2026-04-24T23:59:59-06:00"}
  ],
  "groupIds": "",
  "machineIds": ""
}
```

Respuesta:

```
{
  "errInfo": "操作成功",
  "errCode": "0000",
  "data": [
    {"saleAmount": 0.0, "saleCount": 0},           ← índice 0 = hoy
    {"saleAmount": 1080.0, "saleCount": 10}      ← índice 1 = mes
  ]
}
```

7.5 Estructura de pedidos

orders[].commodityOrderDetailDtos[0]:

- goodsName: nombre del producto
- goodsTypeName: categoría
- singleAmount: precio unitario
- orderNum: cantidad ordenada
- successNum: >0 = Entregado
- failureNum: >0 = Fallo
- (ambos 0) = Pendiente

7.6 Productos reales en la máquina

- Paquete sorpresa 3 (\$40)
- Micas para Pokemon (\$120)
- Paquete Sorpresa 4 (\$40)

- Paquetes random pokemon y Yugioh (\$40)
 - Super Paquete Sorpresa (\$400)
 - Paquete Sorpresa BASE (\$250)
 - Tarjetas Pokemon
 - Mica Yugi Oh
 - Mica Pokemon
-

8. ENTREGABLES COMPLETADOS

8.1 App de escritorio Windows (Python + Tkinter)

- **vmc_desktop_v3.py** — Versión actual con diseño BI profesional
- **Características:**
 - Dashboard con KPI cards, gráfica de barras, gráfica de dona
 - Sidebar oscura tipo Triskell con navegación por iconos
 - Pestañas: Dashboard, Máquinas, Control, Pedidos, Artículos vendidos, Catálogo, Reabastecimiento, Log
 - Filtro de artículos vendidos por período (Hoy/Semana/Mes)
 - Solo cuenta ventas "Entregado" (no pendientes ni fallos)
 - Auto-refresh cada 60 segundos
 - Soporte HiDPI/Retina
 - Token en archivo token.txt (se salta login si existe)
 - Detección de token expirado con diálogo de re-autenticación
 - Captura de errores global con ventana de error y archivo error.log
 - Comando copy() para extraer token desde DevTools del navegador
- **Archivos:** vmc_desktop_v3.py + VMC-Desktop-v3.bat + token.txt

8.2 Panel inyectado en navegador

- Panel HTML/CSS/JS que se inyecta directamente en la plataforma web
- Funciona como overlay sobre a.vmc002.csmology.com
- Mismo diseño oscuro con tabs y control remoto
- Se pierde al navegar (necesita re-inyección)

8.3 Documento de arquitectura

- **VMC-Arquitectura-Sistema-Independiente.pdf** — 12 páginas
- Cobre: hardware, arquitectura actual vs propuesta, plan de 4 fases

8.4 Script de diagnóstico

- **VMC-Diagnostico.bat** — Para ejecutar con ADB desde laptop
 - **diagnostico.sh** — Para ejecutar desde USB directamente en la máquina
 - Extrae: info del dispositivo, APKs, puertos serial, procesos, red, screenshot, logs
-

9. BUGS RESUELTOS DURANTE EL PROYECTO

1. CORS bloqueaba app local → resuelto inyectando panel en plataforma original
 2. Mixed content HTTPS→HTTP → resuelto con app de escritorio
 3. Stats devolvían campos equivocados → corregido con estructura real data[0]/data[1]
 4. /api/menus/build devuelve LIST no objeto → wrapper con _list/_ok
 5. clipboard.writeText falla en DevTools → cambiado a copy() de DevTools
 6. KeyError 'expires' → código viejo de RSA login quedó duplicado, limpiado
 7. App se cuelga en "Verificando" → urlopen puede colgarse en Windows, eliminada verificación
 8. Token JWT expira → sistema de token en archivo con detección de expiración
 9. Artículos vendidos contaba "Pendiente" → filtro solo "Entregado"
 10. UI pixeleada → SetProcessDpiAwareness + tk scaling
-

10. PLAN DE IMPLEMENTACIÓN — ESTADO

Fase 1: Extraer APK y descompilar COMPLETADA

- APK extraído y descompilado
- Protocolo SH del dispensador completamente descifrado
- Bytes exactos de envío/recepción confirmados con logs reales
- Protocolo MDB del billetero documentado
- Bridge JS↔Android mapeado
- UI del cliente analizada

Fase 2: Servidor local + Base de datos SIGUIENTE

- Crear servidor FastAPI en Python con SQLite
- Endpoints: ventas, productos, máquina, config
- WebSocket para comandos en tiempo real
- Migrar dashboard desktop para conectarse al servidor local

Fase 3: App Android propia PENDIENTE

- Proyecto Android Studio con módulo serial (libserial_port.so)
- Comunicación con dispensador (/dev/ttyS4, protocolo SH)
- Comunicación con billeteero (/dev/ttyS1, MDB via Wafer)
- Interfaz de compra WebView
- Conexión WiFi al servidor local

Fase 4: Producción y pulido PENDIENTE

- Modo offline
- Reportes avanzados
- Alertas de stock
- Auto-inicio al encender
- Packaging como APK firmado

11. ARCHIVOS IMPORTANTES

En la computadora de Daniel:

```
C:\Users\USER\Desktop\appwindows\  
├─ vmc_desktop_v3.py          ← App de escritorio actual  
├─ VMC-Desktop-v3.bat         ← Launcher  
├─ token.txt                  ← Token JWT activo  
└─ error.log                  ← Log de errores (si existe)  
  
C:\Users\USER\Desktop\APK CHINO ANDROID\  
├─ vending.apk                ← APK original extraído de la máquina  
└─ (archivos .properties)     ← Configuración extraída
```

Archivos de referencia subidos al chat:

- vending.apk (12.6 MB) — APK de csmVending
 - 20250930.txt (15 MB) — Log completo de operación con bytes seriales
 - ModuleConfig-M3_01.properties — Config del hardware
 - AfcConfig-prod.properties — Config de la app
 - EPayment-M3_01.properties — Config de pagos
 - 16 fotos del hardware interno de la máquina
 - dashboard_ejemplos.pdf — Ejemplos de diseño para la UI
-

12. NOTAS TÉCNICAS IMPORTANTES

1. **El captcha es OBLIGATORIO** para login en la plataforma web. No se puede automatizar el login con RSA. La solución es usar token en archivo.
2. **La máquina está en otro sitio** — Daniel tiene que desplazarse físicamente para acceder al hardware. Toda la info ya fue recopilada.
3. **El protocolo SH** es propietario del fabricante ShangHe (尚和). Los comandos fueron descifrados de los logs reales, no de documentación.
4. **La librería serial** del APK es la estándar android-serialport-api de Google (open source), lo cual facilita la replicación.
5. **El password de backend** de la máquina (DEV.BackerPwd):
149787a6b7986f31b3dcc0e4e857cd2a (hash MD5 en la config).
6. **El número de servicio al cliente** (DEV.kefuNumber): 1803002.
7. **Timezone de la máquina**: CDT / CST (-06:00 México).